PENJELASAN PROYEK

BukuBesar — Core Banking

Core banking mini berbasis event: setiap transfer adalah saga lewat Kafka, setiap rupiah masuk buku besar double-entry yang selalu seimbang, dan transfer besar menunggu supervisor.

Raihan Evanza · Fullstack & Mobile Developer · portfolio-raihan-evanza.vercel.app

PERAN

Proyek pribadi · desain sistem, backend, web, mobile, pengujian, deploy demo

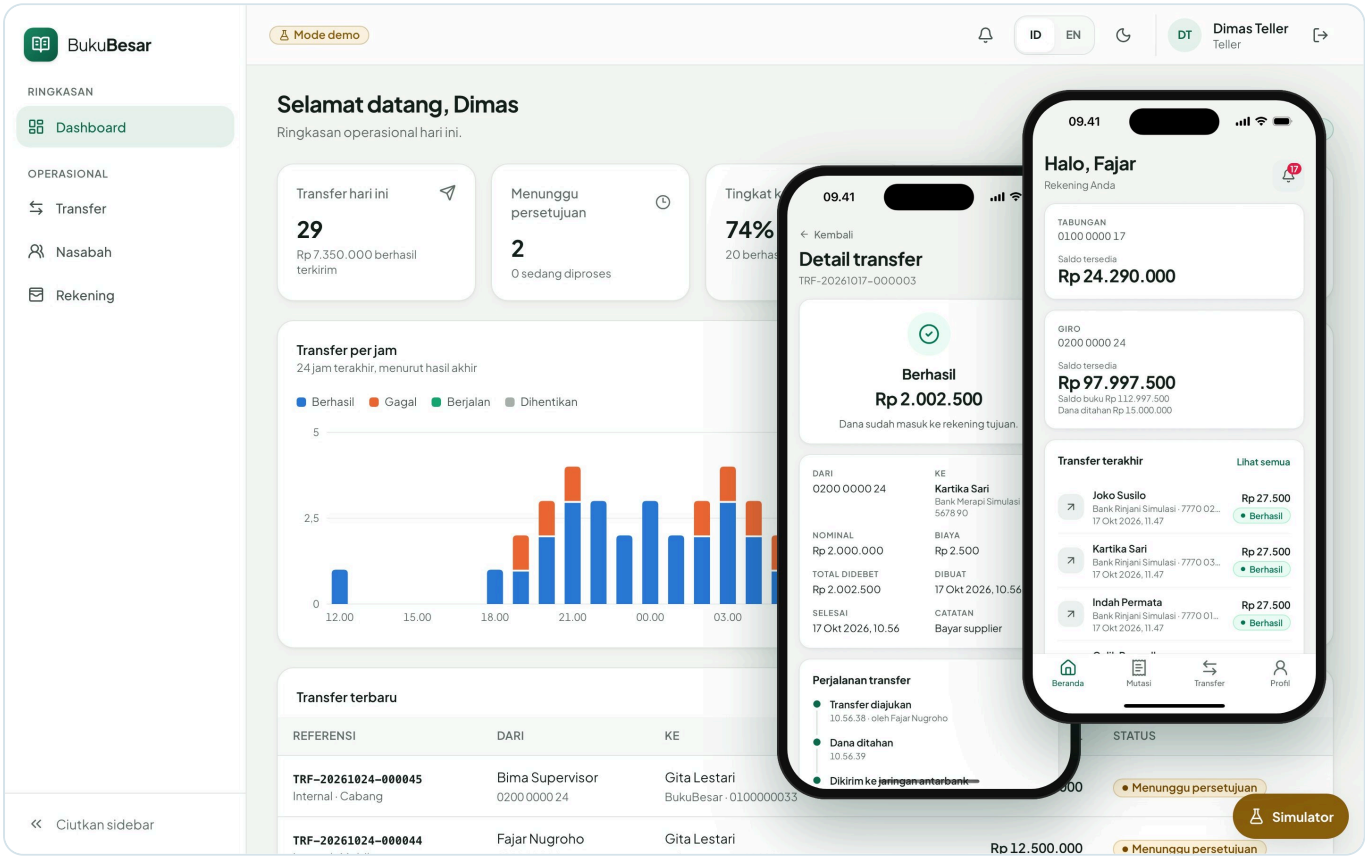
PLATFORM

Web (back office staf) · Android & iOS (aplikasi nasabah) · tiga service Spring Boot

TAUTAN

Demo live: raihan-evanza-bukubesar.vercel.app/login?as=teller

Java 21 · Spring Boot 4.1 · Spring Kafka · PostgreSQL 16 · Flyway · Testcontainers · Vue 3.5 · Pinia · TanStack Query · React Native 0.87 · TypeScript · Tailwind CSS 4 · Rupa Design System · Docker



RINGKASAN

Satu transfer di bank menyentuh banyak sistem: cek limit, menahan dana, mendebit, mengkredit, jaringan antarbank, notifikasi, dan audit. Bila satu langkah gagal di tengah jalan, uang tidak boleh hilang dan tidak boleh terhitung dua kali.

BukuBesar menunjukkan cara membangunnya dengan benar. Tiga service Spring Boot (rekening dan buku besar, transfer, notifikasi), masing-masing dengan database sendiri, saling bicara lewat Kafka. Setiap transfer adalah saga: dana ditahan, transfer besar menunggu supervisor, lalu jurnal double-entry diposting; bila jaringan antarbank menolak atau rekening tujuan dibekukan, dana dikembalikan otomatis (kompensasi).

Klaim pentingnya diuji, bukan hanya diklaim: 50 transfer bersamaan dari satu rekening tidak pernah membuat saldo negatif, service yang dimatikan di tengah saga melanjutkan tanpa kehilangan data, dan 1.700+ event yang diputar

ulang tidak mengubah apa pun. Back office Vue 3, aplikasi nasabah React Native, dan demo yang menjalankan seluruh backend di browser, semuanya memakai Rupa Design System (brand ledger).

FITUR UTAMA

Saga transfer lewat Kafka

transfer-service mengorkestrasi setiap transfer: HoldFunds → FundsHeld → (jaringan antarbank) → PostTransfer → TransferPosted → selesai. Setiap kegagalan setelah dana ditahan melepas hold-nya (kompensasi), termasuk penolakan, pembatalan, dan persetujuan yang kedaluwarsa.

Buku besar double-entry

Setiap transfer menjadi satu jurnal: transfer antarbank tiga baris (rekening nasabah, kewajiban settlement antarbank, pendapatan biaya). Buku besar append-only, jurnal yang tidak seimbang ditolak trigger database, dan neraca saldo selalu membuktikan debit = kredit.

Maker-checker

Transfer mulai Rp10.000.000 dan pembukaan rekening menunggu supervisor. Pembuat pengajuan tidak bisa menyetujuinya sendiri (err_self_approval), dan dua persetujuan bersamaan menghasilkan satu 200 dan satu 409.

Event Inspector

Setiap pesan Kafka, termasuk dead letter topic, dialirkan ke web lewat Server-Sent Events dengan topik, partisi, offset, key, dan payload JSON-nya, sehingga perjalanan saga bisa dilihat langsung.

Back office untuk staf

Dasbor dengan KPI, grafik per jam, dan cek buku besar live; inbox persetujuan dengan hitung mundur; transfer dengan timeline saga; nasabah dan pembukaan rekening; rekening dengan dana ditahan, mutasi bersaldo berjalan, bekukan, dan setor tunai; buku besar, neraca saldo, log audit, dan pengaturan limit.

Aplikasi nasabah

Aplikasi React Native (Android + iOS): rekening di beranda, mutasi dengan filter, transfer internal dan antarbank dengan cek penerima dan konfirmasi, status live dengan timeline saga, batalkan selama menunggu, dan notifikasi.

Lima role, satu peta izin

Teller, Supervisor, Admin, Auditor, dan Nasabah dipetakan ke izin di satu file (Role.java). account-service menandatangani JWT RS256 dan dua service lain memverifikasinya lewat JWKS; kedua aplikasi membaca daftar izin dari /api/auth/me untuk menentukan apa yang ditampilkan.

Simulator, dua bahasa, mode gelap

Di demo, panel Simulator memainkan aplikasi nasabah dengan delapan skenario (internal, antarbank, ditolak jaringan, rekening dibekukan, perlu persetujuan, saldo kurang, kirim dua kali, 20 transfer bersamaan), mengganti persona staf, dan mereset data. Antarmuka tersedia dalam Bahasa Indonesia dan Inggris, terang dan gelap, dan nyaman di ponsel.

ARSITEKTUR & ALUR DATA

Web / app mobile
Back office Vue 3 dan aplikasi nasabah React Native memanggil satu pintu /api.

→

Edge (Caddy)
Meneruskan setiap path ke service pemiliknya.

→

Tiga service Spring Boot
account (auth, rekening, buku besar), transfer (saga, limit, persetujuan), notification (notifikasi, audit, inspector).

→

Kafka (Redpanda)
Command dan event saga lewat outbox; gagal 3× masuk <topic>.DLT.

→

PostgreSQL 16
Satu database per service; tidak ada service yang membaca tabel service lain.

- Event tidak dikirim langsung ke Kafka: service menulis ke tabel outbox dalam transaksi yang sama dengan perubahan datanya, lalu relay mengirimkannya. Data dan event tidak pernah berbeda.
- Setiap consumer mencatat event yang sudah diproses (processed_events), jadi pesan yang terkirim ulang diabaikan. Pesan yang gagal 3 kali (jeda 500 ms) atau tidak bisa dibaca masuk dead letter topic dan terlihat di

Event Inspector.

- Status saga ada di database dan Kafka, tidak di memori, sehingga service yang mati di tengah saga melanjutkan dari posisi terakhir setelah hidup lagi.
- Di demo Vercel, ketiga service, Kafka, dan database digantikan backend di browser dengan kontrak API dan aturan yang sama (bus mirip Kafka dengan partisi dan offset, SSE), yang menyimpan data di localStorage.

TEKNOLOGI PER LAPISAN

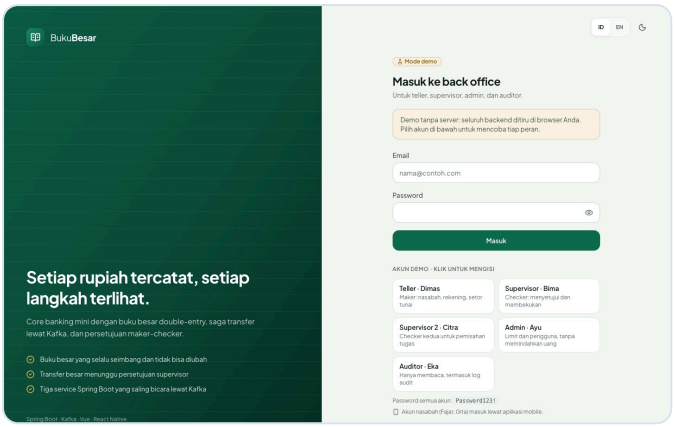
Backend	Java 21 (virtual threads) · Spring Boot 4.1 · Spring Security (OAuth2 resource server, RS256 + JWKS) · Spring Data JPA · Flyway · springdoc-openapi · Maven multi-module
Messaging & data	Spring Kafka · Redpanda / Apache Kafka (Testcontainers) · PostgreSQL 16 (database per service) · Caddy 2 (edge)
Web	Vue 3.5 · TypeScript · Vite · Pinia · TanStack Query · VeeValidate + yup · vue-i18n · Tailwind CSS 4 · Rupa tokens (brand ledger) · Vercel
Mobile	React Native 0.87 (New Architecture, Hermes) · React Navigation 7 · Redux Toolkit + RTK Query · Keychain / Keystore · @rupa-ui/react-native
Pengujian	JUnit 5 + Mockito + AssertJ · Testcontainers (PostgreSQL, Kafka) · Vitest · Jest · e2e + chaos scripts (Node) · GitHub Actions

KEPUTUSAN TEKNIS

1. **Buku besar yang tidak bisa diubah**
   Baris jurnal hanya bisa ditambah: trigger database menolak UPDATE/DELETE dan jurnal yang debit-kreditnya tidak sama. Koreksi dilakukan dengan jurnal baru, persis seperti akuntansi sungguhan, dan saldo rekening selalu sama dengan jumlah barisnya.
2. **Saldo tidak pernah negatif, bahkan saat balapan**
   Setiap hold adalah UPDATE … WHERE available >= amount, ditambah check constraint saldo ≥ 0 dan unique constraint satu hold dan satu jurnal per transfer. Diuji dengan 50 transfer bersamaan dari satu rekening: tepat 33 berhasil, 17 gagal karena saldo kurang, dan saldo akhirnya tepat.
3. **Outbox + consumer idempoten**
   Kafka menjamin pengiriman at-least-once, jadi pesan ganda pasti terjadi. Dengan outbox tidak ada event yang hilang, dan dengan processed_events tidak ada yang diproses dua kali. Semua consumer group diputar ulang dari awal (1.700+ pesan) dan tidak ada data yang berubah.
4. **Retry yang aman lewat Idempotency-Key**
   Transfer dan setor tunai wajib membawa Idempotency-Key yang disimpan bersama hash request. Key dan body yang sama mengembalikan transfer yang sama; key sama dengan body berbeda ditolak 422. Aplikasi mobile bisa mengulang request saat sinyal putus tanpa risiko transfer ganda.
5. **Database per service dan Kafka sungguhan di test**
   Tidak ada service yang membaca tabel service lain; data lintas service hanya lewat event atau API. Lokal memakai Redpanda supaya ringan di laptop 8 GB, tetapi test integrasi menjalankan alur buku besar di Apache Kafka lewat Testcontainers untuk membuktikan kodenya tidak bergantung pada Redpanda.

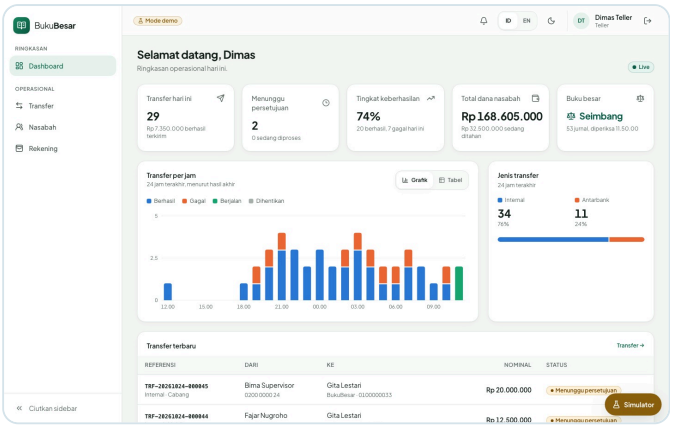
PANDUAN PENGGUNAAN

Langkah-langkah ini sama dengan urutan di video tutorial.



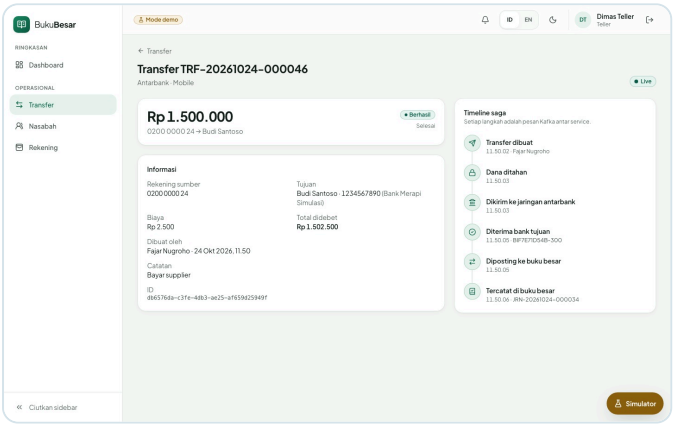
1 Masuk ke back office

Buka demo, klik kartu Teller · Dimas untuk mengisi email dan password otomatis, lalu klik Masuk. Ada lima akun staf (Teller, Supervisor, Supervisor 2, Admin, Auditor) dengan password Password123!; nasabah memakai aplikasi mobile. Tautan /login?as=teller langsung masuk sebagai Dimas.



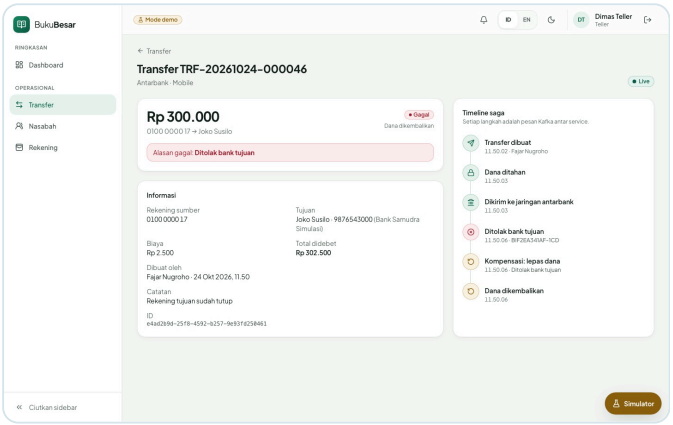
2 Lihat dasbor

Kartu di atas menunjukkan transfer hari ini, yang menunggu persetujuan, tingkat keberhasilan, total dana nasabah, dan status buku besar (Seimbang). Di bawahnya ada grafik transfer per jam menurut hasil akhir, jenis transfer, dan transfer terbaru. Semua berubah live tanpa refresh.



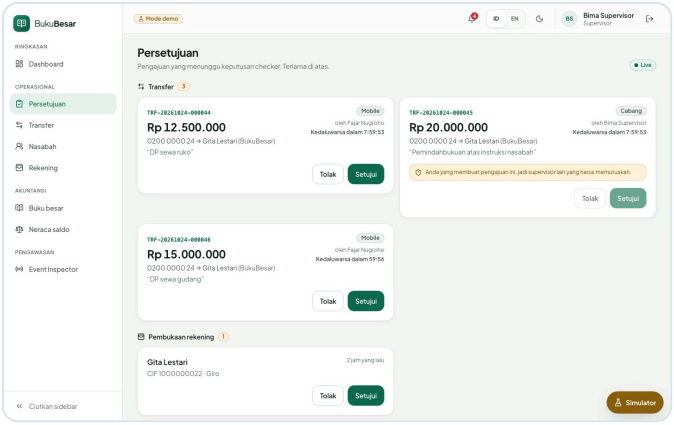
3 Kirim transfer dari Simulator

Klik Simulator di kanan bawah. Panel ini memainkan aplikasi nasabah: jalankan Antarbank Rp1.500.000, lalu buka transfernya. Timeline saga menunjukkan setiap pesan Kafka: transfer dibuat, dana ditahan, dikirim ke jaringan antarbank, diterima bank tujuan, diposting, dan tercatat di buku besar.



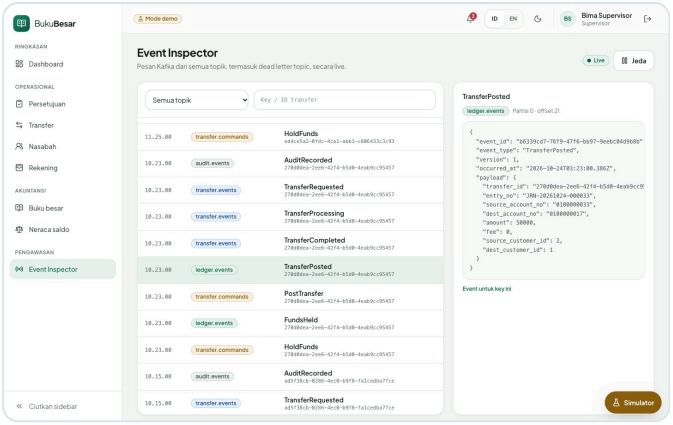
4 Lihat kompensasi

Jalankan Antarbank ke rekening tutup. Jaringan antarbank menolaknya, transfer menjadi Gagal dengan alasan Ditolak bank tujuan, dan saga menjalankan kompensasi: hold dilepas dan dana dikembalikan ke rekening nasabah. Skenario Ke rekening Hana (dibekukan) menunjukkan hal yang sama untuk posting yang ditolak.



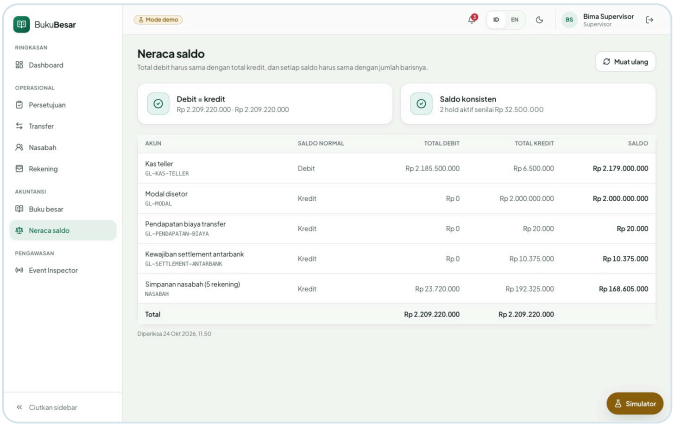
5 Setujui sebagai supervisor

Jalankan Transfer Rp15.000.000: dana ditahan dan transfer menunggu supervisor. Ganti persona ke Supervisor (Bima) di Simulator, lalu buka Persetujuan. Transfer cabang Rp20.000.000 yang dibuat Bima sendiri terkunci (maker-checker), jadi setujui transfer Fajar; catatan persetujuan opsional.



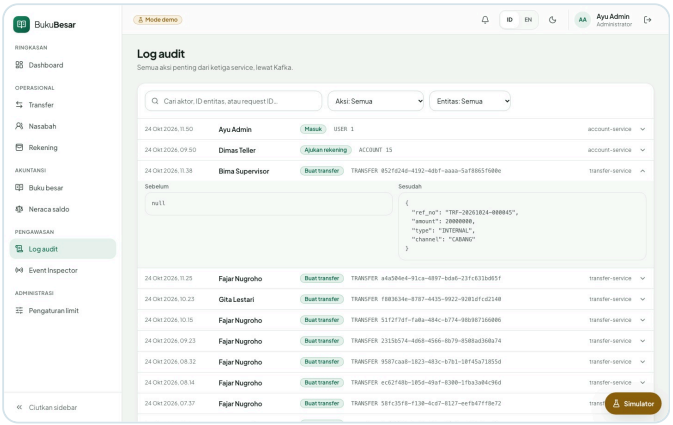
6 Ikuti pesan di Event Inspector

Buka Event Inspector. Setiap pesan Kafka muncul live dengan topik (transfer.commands, ledger.events, audit.events, dan dead letter topic); klik satu baris, misalnya TransferPosted, untuk melihat partisi, offset, dan payload JSON-nya. Filter topik dan cari berdasarkan key atau ID transfer.



7 Periksa buku besar dan neraca saldo

Di Simulator, jalankan 20 transfer bersamaan dari Gita: sebagian gagal, saldo tidak pernah negatif. Lalu buka Neraca saldo: total debit sama dengan total kredit dan setiap saldo konsisten dengan jumlah barisnya. Buka Buku besar dan klik satu jurnal untuk melihat baris debit dan kreditnya.



8 Baca log audit, ganti tema dan bahasa

Ganti persona ke Admin (Ayu) dan buka Log audit: setiap aksi dari ketiga service tercatat dengan nilai sebelum dan sesudah. Gunakan ID/EN dan ikon bulan di header untuk mengganti bahasa dan tema. Reset data demo di Simulator mengembalikan data ke kondisi awal.

KETERBATASAN & CATATAN

- Demo live berjalan tanpa server: ketiga service, Kafka, dan database ditiru oleh backend TypeScript di browser yang menyimpan data di localStorage masing-masing pengunjung. Kontrak API dan aturannya sama, tanpa JVM atau Kafka sungguhan.
- Backend asli (tiga JVM, Redpanda, PostgreSQL) hanya berjalan lokal lewat docker compose; belum ada backend yang di-hosting. Klaim konkurensi, kompensasi, dan replay diuji terhadap stack lokal ini (36 test backend, 20 cek e2e, skrip chaos).
- Jaringan antarbank adalah simulator, bukan integrasi BI-FAST atau switching sungguhan.
- Aplikasi nasabah butuh backend lokal yang berjalan dan belum dirilis ke Play Store/App Store; project native-nya masih memakai nama boilerplate.

RENCANA BERIKUTNYA

- Tarik tunai dengan persetujuan, PIN transaksi di aplikasi mobile, dan notifikasi push.
- Penampil dead letter topic dengan replay, bunga akhir hari (Spring Batch), dan tracing OpenTelemetry.